

React: useState, списки и формы

Методичка для студентов
2 пары по 80 минут

Главная идея: В React мы не меняем HTML напрямую. Мы меняем данные. Когда данные меняются, React сам обновляет интерфейс.

Краткая карта урока

Пара	Время	Тема	Результат
1	0-20 мин	Повторение JSX, компонентов, props, структуры Vite	Студенты вспоминают, как строится статичный интерфейс
1	20-35 мин	События в React	Понимают onClick и передачу функций
1	35-65 мин	useState подробно	Умеют создавать состояние и менять интерфейс
1	65-80 мин	Мини-практика	Закрепляют счетчик, переключатели, boolean state
2	0-10 мин	Повторение useState	Уточняют ключевые ошибки
2	10-35 мин	Списки и map	Умеют выводить массивы в JSX
2	35-55 мин	Формы и input	Понимают value, onChange, event.target.value
2	55-80 мин	Мини-проект	Собирают каталог курсов с добавлением и удалением

Что нужно знать до урока

- Что такое компонент и зачем его выносить отдельно.
- Как работает JSX и почему JavaScript-выражения пишутся в фигурных скобках.
- Что такое props и как родительский компонент передает данные дочернему.
- Где находятся App.jsx и main.jsx в Vite-проекте.
- Как запускать проект через npm run dev.

Пара 1. React становится интерактивным

0-20 минут. Повторение прошлого урока

На прошлом уроке интерфейс был статичным: мы создавали компоненты, передавали данные через props и выводили их в JSX. Сейчас важно быстро повторить фундамент, потому что useState, списки и формы будут строиться поверх этих понятий.

1.1. Структура Vite-проекта

```
src/  
  main.jsx  
  App.jsx  
  index.css  
  components/  
    CourseCard.jsx
```

Файл main.jsx является входной точкой приложения. В нем React берет компонент App и монтирует его в HTML-элемент с id="root".

```
import React from "react"  
import ReactDOM from "react-dom/client"  
import App from "./App.jsx"  
import "./index.css"  
  
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
)
```

Главная строка здесь - <App />. Это запуск главного компонента приложения. Вся дальнейшая работа обычно начинается именно в App.jsx или в компонентах, которые подключаются в App.jsx.

1.2. Повторение JSX

JSX похож на HTML, но внутри него можно использовать JavaScript. Для вставки переменных, выражений и условий используются фигурные скобки.

```
function App() {  
  const name = "Али"  
  const age = 18  
  
  return (  
    <>  
      <h1>Привет, {name}</h1>  
      <p>Через год тебе будет {age + 1}</p>  
    </>  
  )  
}  
  
export default App
```

Важно: JSX должен возвращать один общий родительский элемент. Поэтому часто используют фрагмент <>...</>.

1.3. Повторение компонентов и props

Компонент - это переиспользуемая часть интерфейса. Props - это данные, которые родительский компонент передает дочернему компоненту.

```
function CourseCard({ title, price }) {
```

```

    return (
      <div>
        <h2>{title}</h2>
        <p>Цена: {price} ₴</p>
      </div>
    )
  }

function App() {
  return (
    <>
      <h1>Список курсов</h1>
      <CourseCard title="HTML" price={20000} />
      <CourseCard title="CSS" price={25000} />
      <CourseCard title="JavaScript" price={40000} />
    </>
  )
}

export default App

```

В этом коде App передает данные в CourseCard. Компонент CourseCard не хранит эти данные сам, он просто получает их и отображает.

Понятие	Смысл	Пример
Компонент	Отдельный кусок интерфейса	CourseCard
Props	Данные, пришедшие снаружи	title, price
JSX	Синтаксис разметки внутри JavaScript	{title}
App	Главный компонент приложения	<App />

Переход к новой теме: Пока интерфейс только показывает данные. Сегодня мы научимся делать так, чтобы пользователь мог нажимать, вводить, добавлять и удалять данные.

20-35 минут. События в React

Событие - это действие пользователя: клик, ввод текста, отправка формы, наведение мыши. В React события записываются через camelCase: onClick, onChange, onSubmit.

Обычный HTML	React
onclick	onClick
onchange	onChange
onsubmit	onSubmit
onmouseover	onMouseOver

Первый пример onClick

```

function App() {
  function sayHello() {
    alert("Привет из React")
  }

  return (
    <>
      <h1>События в React</h1>
      <button onClick={sayHello}>Нажми меня</button>
    </>
  )
}

```

```

    </>
  )
}

export default App

```

onClick={sayHello} означает: когда пользователь нажмет на кнопку, React вызовет функцию sayHello.

Частая ошибка: Нельзя писать onClick={sayHello()}, если мы не хотим вызвать функцию сразу при рендере. Скобки вызывают функцию немедленно.

```

// Неправильно: функция вызовется сразу
<button onClick={sayHello()}>Нажми</button>

// Правильно: функция вызовется только при клике
<button onClick={sayHello}>Нажми</button>

```

Передача аргумента в обработчик

Если нужно передать аргумент, используется стрелочная функция. Это нужно, чтобы функция не вызвалась сразу.

```

function App() {
  function sayName(name) {
    alert("Привет, " + name)
  }

  return (
    <>
      <button onClick={() => sayName("Али")}>Поздороваться</button>
    </>
  )
}

export default App

```

35-65 минут. useState подробно

useState - это один из главных инструментов React. Он позволяет компоненту хранить данные, которые меняются во время работы приложения.

- Счетчик лайков хранится в state.
- Текст, введенный в input, хранится в state.
- Список задач хранится в state.
- Статус "меню открыто/закрыто" хранится в state.
- Статус "пользователь вошел/не вошел" хранится в state.

Props против State

Props	State
Приходят в компонент снаружи	Создаются внутри компонента
Передаются от родителя к ребенку	Хранятся в самом компоненте
Компонент не должен менять props напрямую	Компонент меняет state через set-функцию
Нужны для передачи данных	Нужен для интерактивности

Первый счетчик

```
import { useState } from "react"

function App() {
  const [count, setCount] = useState(0)

  function increase() {
    setCount(count + 1)
  }

  return (
    <>
      <h1>Счетчик: {count}</h1>
      <button onClick={increase}>Увеличить</button>
    </>
  )
}

export default App
```

Разбор строки useState

```
const [count, setCount] = useState(0)
```

- count - текущее значение состояния.
- setCount - функция, которая меняет значение count.
- useState(0) - начальное значение. В начале count равен 0.

Общая схема выглядит так:

```
const [значение, функцияДляИзменения] = useState(начальноеЗначение)
```

Почему обычная переменная не подходит

```
function App() {
  let count = 0

  function increase() {
    count = count + 1
    console.log(count)
  }

  return (
    <>
      <h1>Счетчик: {count}</h1>
      <button onClick={increase}>Увеличить</button>
    </>
  )
}

export default App
```

В консоли count будет увеличиваться, но на странице число останется 0. Причина простая: React не следит за обычными переменными. React перерисовывает компонент, когда state меняется через set-функцию.

Правило: Нужно менять не обычную переменную, а state через setCount, setName, setCourses и другие set-функции.

Полный счетчик с тремя кнопками

```
import { useState } from "react"

function App() {
  const [count, setCount] = useState(0)

  function increase() {
    setCount(count + 1)
  }

  function decrease() {
    setCount(count - 1)
  }

  function reset() {
    setCount(0)
  }

  return (
    <>
      <h1>Счетчик: {count}</h1>
      <button onClick={increase}>+</button>
      <button onClick={decrease}>-</button>
      <button onClick={reset}>Сбросить</button>
    </>
  )
}

export default App
```

useState с текстом

```
import { useState } from "react"

function App() {
  const [name, setName] = useState("Али")

  function changeName() {
    setName("Мария")
  }

  return (
    <>
      <h1>Привет, {name}</h1>
      <button onClick={changeName}>Изменить имя</button>
    </>
  )
}
```

```
export default App
```

Здесь state хранит строку. После клика значение name меняется с "Али" на "Мария", и React обновляет интерфейс.

useState с boolean

```
import { useState } from "react"

function App() {
  const [isVisible, setIsVisible] = useState(true)

  function toggleText() {
    setIsVisible(!isVisible)
  }

  return (
    <>
      <button onClick={toggleText}>Показать / скрыть</button>
      {isVisible && <p>Этот текст можно скрывать</p>}
    </>
  )
}

export default App
```

Выражение !isVisible означает "противоположное значение". Если было true, станет false. Если было false, станет true.

65-80 минут. Практика первой пары

Формат: Студенты выполняют задания самостоятельно. Преподаватель ходит по аудитории, проверяет ошибки с импортом useState, onClick и set-функциями.

Практика 1. Счетчик

- Создайте state count.
- Сделайте кнопки +, -, Сбросить.
- Если count больше 10, покажите текст "Слишком много".
- Если count меньше 0, покажите текст "Отрицательное число".

Практика 2. Переключатель текста

- Создайте state isVisible.
- Сделайте кнопку Показать/скрыть.
- При клике текст должен появляться и исчезать.

Практика 3. Статус пользователя

- Создайте state isLoggedIn.
- Если пользователь вошел - показать "Добро пожаловать".
- Если не вошел - показать "Пожалуйста, войдите".
- Кнопка должна менять состояние входа.

Пара 2. Списки, формы и мини-проект

0-10 минут. Быстрое повторение useState

Перед переходом к спискам и формам нужно быстро повторить, как React обновляет интерфейс.

```
import { useState } from "react"

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>+</button>
    </>
  )
}

export default App
```

- Что хранит count?
- Что делает setCount?
- Почему нельзя писать count = count + 1?
- Почему onClick получает функцию, а не результат функции?

10-35 минут. Списки и map

В реальных React-приложениях данные редко пишут вручную в JSX. Обычно данные приходят массивом: из базы данных, API, локального состояния или файла. React должен уметь превратить массив данных в список компонентов.

Проблема ручной отрисовки

```
function App() {
  return (
    <>
      <h1>Курсы</h1>

      <div>
        <h2>HTML</h2>
        <p>Цена: 20000 ₺</p>
      </div>

      <div>
        <h2>CSS</h2>
        <p>Цена: 25000 ₺</p>
      </div>

      <div>
        <h2>JavaScript</h2>
        <p>Цена: 40000 ₺</p>
      </div>
    </>
  )
}
```



```
export default App
```

Такой подход неудобен: много повторения, сложно добавлять и удалять элементы, невозможно удобно работать с данными с сервера.

Массив строк и `map`

```
function App() {
  const courses = ["HTML", "CSS", "JavaScript", "React"]

  return (
    <>
      <h1>Курсы</h1>

      {courses.map((course) => (
        <p key={course}>{course}</p>
      ))}
    </>
  )
}

export default App
```

Метод `map` проходит по каждому элементу массива и возвращает новый JSX-элемент.

```
"HTML"      -> <p>HTML</p>
"CSS"       -> <p>CSS</p>
"JavaScript" -> <p>JavaScript</p>
"React"     -> <p>React</p>
```

Массив объектов

```
function App() {
  const courses = [
    { id: 1, title: "HTML", price: 20000 },
    { id: 2, title: "CSS", price: 25000 },
    { id: 3, title: "JavaScript", price: 40000 },
    { id: 4, title: "React", price: 50000 },
  ]

  return (
    <>
      <h1>Курсы</h1>

      {courses.map((course) => (
        <div key={course.id}>
          <h2>{course.title}</h2>
          <p>Цена: {course.price} ₸</p>
        </div>
      ))}
    </>
  )
}

export default App
```

Один объект `course` содержит `id`, `title` и `price`. В JSX мы обращаемся к ним через точку: `course.title`, `course.price`.

Зачем нужен `key`

`key` - это специальный атрибут, который нужен React для списков. Он помогает React понять, какой элемент был добавлен, удален или изменен.

```
{courses.map((course) => (  
  <div key={course.id}>  
    <h2>{course.title}</h2>  
  </div>  
))}
```

Правило: Лучше использовать уникальный `id`. Index можно использовать только в очень простых списках, которые не сортируются и не удаляются.

35-55 минут. Списки + компоненты + props

Теперь связываем прошлый урок и новую тему. App хранит массив данных, `map` проходит по массиву, а отдельный компонент отвечает за внешний вид одной карточки.

```
function CourseCard({ course }) {  
  return (  
    <div>  
      <h2>{course.title}</h2>  
      <p>Цена: {course.price} ₸</p>  
    </div>  
  )  
}  
  
function App() {  
  const courses = [  
    { id: 1, title: "HTML", price: 20000 },  
    { id: 2, title: "CSS", price: 25000 },  
    { id: 3, title: "JavaScript", price: 40000 },  
    { id: 4, title: "React", price: 50000 },  
  ]  
  
  return (  
    <>  
      <h1>Курсы</h1>  
  
      {courses.map((course) => (  
        <CourseCard key={course.id} course={course} />  
      ))}  
    </>  
  )  
}  
  
export default App
```

Часть	Ответственность
App	Хранит массив <code>courses</code>
<code>map</code>	Проходит по массиву и создает карточки
CourseCard	Отвечает за отображение одного курса

props	Передают course из App в CourseCard
-------	-------------------------------------

55-65 минут. Формы и input

Форма нужна, чтобы пользователь мог вводить данные. В React значение input обычно хранится в state. Такой input называется controlled input.

Первый input

```
import { useState } from "react"

function App() {
  const [title, setTitle] = useState("")

  return (
    <>
      <h1>Название: {title}</h1>

      <input
        type="text"
        placeholder="Введите название"
        value={title}
        onChange={(event) => setTitle(event.target.value)}
      />
    </>
  )
}

export default App
```

value={title} означает, что значение input берется из state. onChange срабатывает при каждом изменении текста. event.target.value - это то, что сейчас написано в input.

```
Пользователь вводит текст
↓
onChange получает событие
↓
event.target.value содержит новый текст
↓
setTitle обновляет state
↓
React обновляет интерфейс
```

Форма с двумя input

```
import { useState } from "react"

function App() {
  const [title, setTitle] = useState("")
  const [price, setPrice] = useState("")

  function handleSubmit(event) {
    event.preventDefault()

    console.log("Название:", title)
    console.log("Цена:", price)
  }
}
```

```

return (
  <>
    <h1>Добавление курса</h1>

    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Название курса"
        value={title}
        onChange={(event) => setTitle(event.target.value)}
      />

      <input
        type="number"
        placeholder="Цена курса"
        value={price}
        onChange={(event) => setPrice(event.target.value)}
      />

      <button type="submit">Добавить</button>
    </form>
  </>
)
}

export default App

```

event.preventDefault(): Отключает стандартную перезагрузку страницы при отправке формы. Для React-приложений это почти всегда нужно.

65-80 минут. Мини-проект: каталог курсов

Итоговый мини-проект объединяет все темы урока: useState, списки, map, props, формы, добавление и удаление.

Функционал проекта

- Показать список курсов.
- Каждый курс вывести через отдельный компонент CourseCard.
- Добавить форму с названием и ценой курса.
- При отправке формы добавить новый курс в список.
- После добавления очистить input.
- Запретить добавление пустых полей.
- Добавить кнопку удаления курса.
- Если курсов нет, показать текст "Курсов пока нет".

Финальный код проекта

```

import { useState } from "react"

function CourseCard({ course, onDelete }) {
  return (
    <div>
      <h2>{course.title}</h2>
      <p>Цена: {course.price} ₸</p>

```

```

        <button onClick={() => onDelete(course.id)}>
            Удалить
        </button>
    </div>
)
}

function App() {
    const [courses, setCourses] = useState([
        { id: 1, title: "HTML", price: 20000 },
        { id: 2, title: "CSS", price: 25000 },
        { id: 3, title: "JavaScript", price: 40000 },
    ])

    const [title, setTitle] = useState("")
    const [price, setPrice] = useState("")

    function addCourse(event) {
        event.preventDefault()

        if (title.trim() === "" || price.trim() === "") {
            alert("Заполните все поля")
            return
        }

        const newCourse = {
            id: Date.now(),
            title: title,
            price: price,
        }

        setCourses([...courses, newCourse])

        setTitle("")
        setPrice("")
    }

    function deleteCourse(id) {
        setCourses(courses.filter((course) => course.id !== id))
    }

    return (
        <>
            <h1>Каталог курсов</h1>

            <form onSubmit={addCourse}>
                <input
                    type="text"
                    placeholder="Название курса"
                    value={title}
                    onChange={(event) => setTitle(event.target.value)}
                />

                <input
                    type="number"
                    placeholder="Цена курса"
                    value={price}
                    onChange={(event) => setPrice(event.target.value)}

```

```

    />

    <button type="submit">Добавить курс</button>
  </form>

  <h2>Список курсов</h2>

  {courses.length === 0 ? (
    <p>Курсов пока нет</p>
  ) : (
    courses.map((course) => (
      <CourseCard
        key={course.id}
        course={course}
        onDelete={deleteCourse}
      />
    ))
  )}
</>
)
}

export default App

```

Подробный разбор мини-проекта

1. Компонент CourseCard

```
function CourseCard({ course, onDelete }) {
```

Компонент принимает два props: `course` - объект курса, `onDelete` - функция удаления. Сам компонент не хранит список курсов. Он только показывает один курс и вызывает функцию удаления при клике.

```

<button onClick={() => onDelete(course.id)}>
  Удалить
</button>

```

Стрелочная функция нужна, потому что мы передаем аргумент `course.id`. Если написать `onClick={onDelete(course.id)}`, функция вызовется сразу.

2. State со списком курсов

```

const [courses, setCourses] = useState([
  { id: 1, title: "HTML", price: 20000 },
  { id: 2, title: "CSS", price: 25000 },
  { id: 3, title: "JavaScript", price: 40000 },
])

```

`courses` - текущий массив курсов. `setCourses` - функция для замены массива на новый. В React нельзя напрямую мутировать массив через `push`, `splice` и подобные методы.

3. State для формы

```
const [title, setTitle] = useState("")
```

```
const [price, setPrice] = useState("")
```

Каждый input получает свой state. Это позволяет React контролировать содержимое input и использовать эти данные при отправке формы.

4. Добавление курса

```
function addCourse(event) {
  event.preventDefault()

  if (title.trim() === "" || price.trim() === "") {
    alert("Заполните все поля")
    return
  }

  const newCourse = {
    id: Date.now(),
    title: title,
    price: price,
  }

  setCourses([...courses, newCourse])

  setTitle("")
  setPrice("")
}
```

- `event.preventDefault()` отключает перезагрузку страницы.
- Проверка `trim()` не дает добавить пустой курс.
- `Date.now()` создает учебный уникальный id.
- `setCourses([...courses, newCourse])` создает новый массив.
- `setTitle("")` и `setPrice("")` очищают поля формы.

5. Удаление курса

```
function deleteCourse(id) {
  setCourses(courses.filter((course) => course.id !== id))
}
```

`filter` возвращает новый массив. В этот массив попадут все курсы, кроме курса с `id`, который нужно удалить.

```
// Было:
[
  { id: 1, title: "HTML" },
  { id: 2, title: "CSS" },
  { id: 3, title: "React" },
]

// Удаляем id = 2

// Стало:
[
  { id: 1, title: "HTML" },
  { id: 3, title: "React" },
]
```

Типичные ошибки и как их исправить

Ошибка	Почему плохо	Правильно
useState is not defined	Забыли импортировать useState	import { useState } from "react"
onClick={func()}	Функция вызывается сразу	onClick={func} или onClick={() => func(id)}
array.push(item)	Мутация старого массива	setArray([...array, item])
Нет key в map	React не понимает стабильность элементов	key={item.id}
Форма перезагружает страницу	Не отключено стандартное поведение	event.preventDefault()
Input не печатает текст	Есть value, но нет onChange	Добавить onChange и setState

Итоговая схема React-мышления

```

Пользователь нажимает кнопку / вводит текст
↓
Срабатывает событие onClick / onChange / onSubmit
↓
Вызывается set-функция: setCount, setTitle, setCourses
↓
State изменяется
↓
React заново вызывает компонент
↓
JSX строится заново
↓
Интерфейс обновляется

```

Главный вывод: В React не нужно вручную искать элементы через document.querySelector и менять innerHTML. Нужно менять данные, а React сам обновит интерфейс.

Контрольные вопросы для закрепления

1. Что такое state в React?
2. Чем state отличается от props?
3. Что возвращает useState?
4. Почему нельзя менять state напрямую?
5. Что делает setCount?
6. Зачем нужен onClick?
7. Когда нужна стрелочная функция в onClick?
8. Что делает метод map?
9. Зачем нужен key?
10. Что такое controlled input?
11. Что находится в event.target.value?
12. Зачем нужен event.preventDefault()?
13. Как добавить новый элемент в массив state?

14. Как удалить элемент из массива state?

Домашнее задание

Дедлайн: 10 дней с момента выдачи задания. Работы сдаются в виде ссылки на GitHub-репозиторий или архива проекта.

Домашние задания несложные. Главная цель - закрепить `useState`, `map`, `props`, формы, добавление и удаление элементов. Можно выбрать все 5 заданий или выполнить их последовательно как мини-тренажеры.

Домашка 1. Счетчик оценок

- Создайте приложение "Счетчик оценок".
- На экране должно быть число `score`.
- Кнопка `+1` увеличивает `score`.
- Кнопка `-1` уменьшает `score`.
- Кнопка Сбросить возвращает `score` к 0.
- Если `score` больше или равен 10, показать текст "Отличный результат".

Домашка 2. Список любимых фильмов

- Создайте массив фильмов в `state`.
- У каждого фильма должны быть `id`, `title`, `year`.
- Выведите фильмы через `map`.
- Создайте компонент `MovieCard`.
- Передавайте фильм в `MovieCard` через `props`.
- Добавьте кнопку удаления фильма.

Домашка 3. Добавление студента

- Создайте список студентов.
- У студента должны быть `id`, `name`, `age`, `group`.
- Создайте форму с `input`: имя, возраст, группа.
- После отправки формы студент добавляется в список.
- Пустые поля добавлять нельзя.
- После добавления `input` должны очищаться.

Домашка 4. Todo List

- Создайте `input` для новой задачи.
- Создайте кнопку Добавить.
- Задачи должны храниться в `state`.
- Выведите задачи через `map`.
- У каждой задачи должна быть кнопка Удалить.
- Если задач нет, показать текст "Задач пока нет".

Домашка 5. Каталог книг

- Создайте список книг.
- У книги должны быть `id`, `title`, `author`, `price`.
- Сделайте компонент `BookCard`.
- Выведите список книг через `map`.
- Добавьте форму: название, автор, цена.
- Добавляйте новую книгу в список.

- Добавьте удаление книги.
- Проверьте, чтобы пустые поля не добавлялись.

Мини-критерии проверки домашнего задания

Критерий	Баллы
Проект запускается без ошибок	2
Корректно используется useState	2
Список выводится через map и key	2
Есть форма с controlled input	2
Есть добавление и удаление элементов	2
Итого	10

Что будет дальше

После этого урока логично продолжить тему на проекте Todo List с фильтрацией и сохранением в localStorage. Это закрепит useState, списки и формы, а затем можно переходить к useEffect и запросам к API.